

Tydenso

Symbolic tensor algebra inside Typst

User manual · version 0.1.0

Build indexed tensors as readable Typst values, transform them with Idenso, and print them with Spenso's own notation.

[Repository](#) · [MIT license](#) · [Spenso Python API](#) · [Idenso Python API](#)

Contents

1	Start with a contraction	3
1.1	Installation	3
1.2	How this corresponds to the Python API	3
2	Build tensor expressions	3
2.1	Representations and slots are data	3
2.2	Tensor names carry symmetry	4
2.3	Write tensor algebra as math	4
3	Transform tensors	4
3.1	Contract a metric chain	5
3.2	Know which family to call	5
4	Print and inspect	5
4.1	Use the Spenso printer directly	5
4.2	Inspect the Atom as CBOR data	6
5	Work alongside Tymbolica	6
6	API reference	7
6.1	Tensor construction	7
6.2	Expression construction	13
6.3	Printing and inspection	15
6.4	Metric and index transformations	17
6.5	Dirac and color transformations	19
6.6	Advanced configuration	19
7	Compatibility and licensing	20
8	Acknowledgements	20

1 Start with a contraction

Tydenso is the tensor-focused companion to Tymbolica. It has its own package, manual, printer, and WebAssembly engine. You do not need Tymbolica to construct, simplify, inspect, or display a tensor expression.

The first example contracts a Minkowski metric with a vector. A representation is an ordinary Typst dictionary; slot turns an index label into another dictionary; vector creates a callable rank-one tensor name.

```
#let V = mink(4)
#let mu = slot(V, "mu")
#let nu = slot(V, "nu")
#let p = vector("p")

#let before = math($#metric(V, mu, nu) #p(nu)$)
#let after = simplify-metrics(before)

$
#to-typst(before)
quad arrow.r.long
#to-typst(after)
$
```

$$p^\nu g^{\mu\nu} \longrightarrow p^\mu$$

The constructor calls already produce readable math. Their hidden, versioned metadata contains the exact Symbolica Atom, so math does not have to infer a tensor from its appearance. It returns Atom bytes for algebra; to-typst renders transformed results.

1.1 Installation

Until Tydenso is published separately, install the tydenso directory from this repository as its own local package. On Linux, place or symlink it at:

```
~/local/share/typst/packages/local/tydenso/0.1.0
```

Then import it independently:

```
#import "@local/tydenso:0.1.0": *
```

From a checkout, #import "path/to/tymbolica/tydenso/lib.typ": * works as well. The package directory already contains its compressed engine and loader.

1.2 How this corresponds to the Python API

The public concepts follow Spenso's Python surface, adapted to what is natural in Typst. Python objects can overload calls; Typst dictionaries cannot, so index construction is the explicit slot(V, "mu"). Tensor names are functions, which keeps the pleasant p(mu) spelling.

Python	Typst
<pre>V = Representation.mink(4) mu = V("mu") nu = V("nu") p = TensorName("p") expr = TensorName.g(mu, nu) * p(nu)</pre>	<pre>#let V = mink(4) #let mu = slot(V, "mu") #let nu = slot(V, "nu") #let p = vector("p") #let expr = math(\$#metric(V, mu, nu) #p(nu)\$)</pre>

Representations and slots stay transparent dictionaries. Symbols and tensor calls are visible Typst math with exact Atom metadata; completed algebraic expressions are Atom bytes.

2 Build tensor expressions

2.1 Representations and slots are data

Built-in constructors cover the representations initialized by Spenso and Idenso: mink, euc, lor, bis, spf, cof, coad, and cos. Use representation when a model introduces another representation.

```
#let color = cof(3)
#let a = slot(color, "a")
#let b-lower = slot(color, "b", dual: true)

#table(
  columns: (auto, lfr),
  inset: 5pt,
  [representation], [#color.name],
  [dimension], [#color.dimension],
  [first index], [#a.index],
  [second index is dual], [#b-lower.dual],
)
```

representation	cof
dimension	3
first index	a
second index is dual	true

Because those values are dictionaries, document code can validate dimensions, generate index families in a loop, or attach its own metadata before asking the plugin to construct an Atom. `dual` - representation returns the paired representation; `metric`, `identity-tensor`, and `flat-tensor` accept either index labels or already constructed slots.

2.2 Tensor names carry symmetry

tensor mirrors the useful part of Spenso's `TensorName`: a name plus optional symmetric, antisymmetric, cyclic, or linear behavior. Symbolica applies those attributes while it constructs the function, not as a later cosmetic step.

```
#let V = euc(3)
#let mu-slot = slot(V, "mu")
#let nu-slot = slot(V, "nu")
#let F = tensor("F", antisymmetric: true)

#let cancellation = math($#F(mu-slot, nu-slot) + #F(nu-slot, mu-slot)$)
$ F^(mu nu) + F^(nu mu) = #to-typst(cancellation) $
```

$$F^{\mu\nu} + F^{\nu\mu} = 0$$

Only one of symmetric, antisymmetric, and cycle-symmetric may be true for one tensor name. The `linear` flag is independent.

2.3 Write tensor algebra as math

Tensor and vector constructors are Typst functions. Interpolate their calls in math with `#F( . . . )`; interpolate scalar symbols with `#mass`. Parsely reads the ordinary arithmetic, while each annotated value contributes its exact Atom.

```
#let V = mink("D")
#let mu = slot(V, "mu")
#let nu = slot(V, "nu")
#let p = vector("p")
#let mass = symbol("m", namespace: "model")

#let shell = math($#metric(V, mu, nu) #p(mu) #p(nu) - #mass^2$)
$ #to-typst(shell) $
```

$$p^\mu p^\nu g^{\mu\nu} - m^2$$

The structural `add`, `mul`, `neg`, `sub`, `div`, and `pow` functions remain useful for generated expressions. They accept Atom bytes, annotated content, numbers, slots, and representation dictionaries. Both paths construct the same Atom; neither reconstructs tensors from printed subscripts.

3 Transform tensors

Idenso provides the domain-specific transformations. The most common ones simplify metrics, gamma matrices, and color structures; selective expanders and index-wrapping functions are available for lower-level workflows.

3.1 Contract a metric chain

The next expression contains two metrics and one vector. Simplifying metrics eliminates both contracted dummy indices in one pass.

```
#let V = mink(4)
#let mu = slot(V, "mu")
#let nu = slot(V, "nu")
#let rho = slot(V, "rho")
#let p = vector("p")

#let chain = mul(
  metric(V, mu, nu),
  metric(V, nu, rho),
  p(rho),
)
#let reduced = simplify-metrics(chain)

$
#to-typst(chain)
quad arrow.r.long
#to-typst(reduced)
$
```

$$p^\rho g^{\mu\nu} g^{\nu\rho} \longrightarrow p^\mu$$

`list-dangling` returns the free indices as Atom payloads. Pass an element to `to-typst`, `to-string`, or `inspect` just like any other expression.

3.2 Know which family to call

Family	Purpose
<code>simplify-metrics</code> , <code>expand-metrics</code> , <code>expand-mink</code> , <code>expand-bis</code>	Metric, Minkowski, and bispinor structure.
<code>simplify-gamma</code> , <code>dirac-adjoint</code>	Dirac chains and conjugation.
<code>simplify-color</code> , <code>expand-color</code>	Color generators and structure constants.
<code>cook-function</code> , <code>cook-indices</code> , <code>wrap-dummies</code> , <code>wrap-indices</code>	Canonical index organization and explicit wrappers.
<code>to-dots</code>	Rewrite supported contractions as dot products.

The mathematical conventions and supported identities track [Idenso's API](#).

4 Print and inspect

4.1 Use the Spenso printer directly

Tydenso does not send tensor output through Tymbolica's general printer. `to-typst-source` uses Symbolica's Typst arithmetic mode together with Spenso's custom tensor notation. `to-string` uses Spenso's compact Symbolica notation.

```
#let V = mink(4)
#let p = vector("p")
#let expression = p(1, slot(V, "mu"))

#let detailed = print-settings(with-dim: true, commas: true)

Rendered: $ #to-typst(expression, settings: detailed) $

Compact Spenso form:
#raw(to-string(expression), block: true)
```

Rendered:

$$p_1^\mu$$

Compact Spenso form:

`p(1,amu)`

Upper and lower indices align in matching columns. A plain self-dual slot is placed on the top row; for a dualizable representation, its dual orientation is placed on the bottom row.

`print-settings` exposes the real `SpensoPrintSettings` switches: `with-dim`, `parens`, `commas`, `index-subscripts`, and `symbol-scripts`. Start from either the `"typst"` or `"compact"` preset and override only what the document needs.

4.2 Inspect the Atom as CBOR data

Atom bytes remain the lossless transformation and interchange format. Before a transformation, annotated constructor content carries those same bytes in metadata. `inspect` accepts either form and decodes it to recursive Typst data. Every node has a `kind`; function nodes also expose their full and short names, arguments, and symmetry flags.

```
#let V = euc(3)
#let A = tensor("A", symmetric: true)
#let expression = A(slot(V, "i"), slot(V, "j"))
#let tree = inspect(expression)

#table(
  columns: (auto, lfr),
  inset: 5pt,
  [node kind], [#tree.kind],
  [function], [#tree.short-name],
  [arguments], [#tree.arguments.len()],
  [symmetric], [#tree.symmetric],
)
```

node kind	function
function	A
arguments	2
symmetric	true

Why keep both forms? CBOR is convenient for layouts, diagnostics, and package-level APIs. Reconstructing algebra from that tree would lose Symbolica state and exact coefficient details, so transformations continue to consume Atom bytes.

5 Work alongside Tymbolica

Tydenso and Tymbolica use the same versioned native Atom export. A package can construct and simplify tensors with Tydenso, then pass the result to Tymbolica for a general algebraic operation. It can also go the other way when a Tymbolica expression uses Spenso's tensor representation.

```
#import "@local/tydenso:0.1.0" as tensors
#import "@local/tymbolica:0.1.0" as algebra

#let V = tensors.mink(4)
#let p = tensors.vector("p")
#let expression = p(tensors.slot(V, "mu"))

#let expanded = algebra.expand(expression)
#tensors.to-typst(expanded)
```

Keep package versions aligned. Matrix payloads from Tymbolica are a different format and are not Tydenso inputs.

6 API reference

The groups below cover the imported top-level API. `init` returns the same surface as a dictionary bound to a selected plugin module.

6.1 Tensor construction

6.1.1 tensor

Construct a named tensor function.

The result is callable with any mixture of slots, representations, scalar values, and compatible Atom payloads. Symmetry flags follow Spenso's `TensorName` model and are registered on the underlying Symbolica symbol.

```
#let V = mink(4)
#let Z = tensor("Z")
#to-tyrst(Z(slot(V, "mu"), slot(V, "nu")))
```

$$Z^{\mu\nu}$$

Parameters

```
tensor(
  name: str,
  namespace: str,
  symmetric: bool,
  antisymmetric: bool,
  cycle-symmetric: bool,
  linear: bool,
  tags: array
) -> function
```

6.1.1.0.1 name str

Tensor name.

6.1.1.0.2 namespace str

Symbol namespace.

Default: "spenso"

6.1.1.0.3 symmetric bool

Make the tensor symmetric under argument permutation.

Default: `false`

6.1.1.0.4 antisymmetric bool

Make the tensor antisymmetric under argument permutation.

Default: `false`

6.1.1.0.5 cycle-symmetric bool

Make the tensor symmetric under cyclic argument permutation.

Default: `false`

6.1.1.0.6 linear bool

Mark the tensor function as linear.

Default: `false`

6.1.1.0.7 tags `array`

Portable semantic labels retained in attached metadata.

Default: `()`

6.1.2 vector

Construct a named rank-one tensor function.

Non-slot arguments are rendered as symbol subscripts in Spenso's Typst mode, while the vector slot is rendered as an abstract index.

```
#let V = mink(4)
#let p = vector("p")
#to-typst(p(1, slot(V, "mu")))
```

$$p_1^\mu$$

Parameters

```
vector(
  name: str,
  namespace: str,
  linear: bool,
  tags: array
) -> function
```

6.1.2.0.1 name `str`

Vector name.

6.1.2.0.2 namespace `str`

Symbol namespace.

Default: `"spenso"`

6.1.2.0.3 linear `bool`

Mark the vector function as linear.

Default: `false`

6.1.2.0.4 tags `array`

Portable semantic labels retained in attached metadata.

Default: `()`

6.1.3 symbol

Construct a symbolic scalar with exact Atom metadata.

Parameters

```
symbol(  
  name: str,  
  namespace: str,  
  tags: array  
) -> content
```

6.1.3.0.1 name str

Symbol name.

6.1.3.0.2 namespace str

Symbol namespace.

Default: "spenso"

6.1.3.0.3 tags array

Portable semantic labels retained in attached metadata.

Default: ()

6.1.4 function

Construct a callable symbolic function with exact Atom metadata.

This is distinct from `symbol`: a Typst content value is not callable. Calling the returned function annotates the complete function call.

Parameters

```
function(  
  name: str,  
  namespace: str,  
  tags: array  
) -> function
```

6.1.4.0.1 name str

Function-head name.

6.1.4.0.2 namespace str

Symbol namespace.

Default: "spenso"

6.1.4.0.3 tags array

Portable semantic labels retained in attached metadata.

Default: ()

6.1.5 representation

Describe a representation and attach its tensor-building methods.

The returned dictionary is inspectable Typst data. Its slot, metric, identity, flat, and dual fields are functions.

Parameters

```
representation(
  name: str,
  dimension: int | str | bytes,
  namespace: str,
  self-dual: bool,
  is-dual: bool,
  dual-name: str | none
) -> dictionary
```

6.1.5.0.1 name str

Symbolic representation name.

6.1.5.0.2 dimension int or str or bytes

Dimension, which may itself be symbolic.

6.1.5.0.3 namespace str

Symbol namespace.

Default: "spenso"

6.1.5.0.4 self-dual bool

Whether the representation is its own dual.

Default: false

6.1.5.0.5 is-dual bool

Whether this value denotes the dual orientation of a non-self-dual representation.

Default: false

6.1.5.0.6 dual-name str or none

Name used by the representation returned from dual.

Default: none

6.1.6 mink

Construct a Minkowski representation.

Parameters

```
mink(dimension) -> dictionary
```

6.1.7 euc

Construct a Euclidean representation.

Parameters

```
euc(dimension) -> dictionary
```

6.1.8 lor

Construct a Lorentz representation.

Parameters

```
lor(dimension) -> dictionary
```

6.1.9 bis

Construct a bispinor representation.

Parameters

```
bis(dimension) -> dictionary
```

6.1.10 spf

Construct a spin-fundamental representation.

Parameters

```
spf(dimension) -> dictionary
```

6.1.11 cof

Construct a color-fundamental representation.

Parameters

```
cof(dimension) -> dictionary
```

6.1.12 coad

Construct a color-adjoint representation.

Parameters

```
coad(dimension) -> dictionary
```

6.1.13 cos

Construct a color-sextet representation.

Parameters

```
cos(dimension) -> dictionary
```

6.1.14 slot

Construct an indexed slot from a representation.

The result is an ordinary dictionary, so its representation, index, and variance remain visible to Typst before an expression is constructed.

Parameters

```
slot(
  representation: dictionary,
  index: int str bytes,
  dual: bool none
) -> dictionary
```

6.1.14.0.1 representation dictionary

Representation dictionary.

6.1.14.0.2 index int or str or bytes

Abstract index label.

6.1.14.0.3 dual bool or none

Wrap the slot with Spensio's dual-index marker. none inherits the representation's is-dual field.

Default: none

6.1.15 metric

Construct the metric tensor for a representation.

Index labels are converted to slots automatically; existing slots pass through unchanged.

Parameters

```
metric(
  representation,
  first,
  second
) -> content
```

6.1.16 identity-tensor

Construct the identity tensor for a representation.

Parameters

```
identity-tensor(
  representation,
  first,
  second
) -> content
```

6.1.17 flat-tensor

Construct Spenso's musical isomorphism tensor for a representation.

Parameters

```
flat-tensor(
    representation,
    first,
    second
) -> content
```

6.1.18 dual-representation

Return the representation paired with this representation.

Parameters

```
dual-representation(representation) -> dictionary
```

6.2 Expression construction

6.2.1 math

Parse Typst math into an exact Symbolica Atom payload.

Annotated values created by `symbol`, `function`, `tensor`, `vector`, and the named tensor constructors are imported from their exact Atom metadata. Use interpolation for callable bindings, for example `#F(mu, nu)`.

Parameters

```
math(
    equation: content,
    grammar: dictionary none,
    namespace: str none
) -> bytes
```

6.2.1.0.1 equation content

Math content, normally written as $\dots$.

6.2.1.0.2 grammar dictionary or none

Parser grammar override. `none` uses the engine grammar.

Default: `none`

6.2.1.0.3 namespace str or none

Namespace for unannotated parsed symbols.

Default: `none`

6.2.2 atom

Convert a supported value or annotated math expression to Atom bytes.

Parameters

```
atom(value) -> bytes
```

6.2.3 add

Add symbolic expressions or constructor values.

Parameters

```
add(..terms) -> bytes
```

6.2.4 mul

Multiply symbolic expressions or constructor values.

Parameters

```
mul(..factors) -> bytes
```

6.2.5 neg

Negate a symbolic expression or constructor value.

Parameters

```
neg(expression) -> bytes
```

6.2.6 sub

Subtract two symbolic expressions or constructor values.

Parameters

```
sub(  
  left,  
  right  
) -> bytes
```

6.2.7 div

Divide two symbolic expressions or constructor values.

Parameters

```
div(  
  numerator,  
  denominator  
) -> bytes
```

6.2.8 pow

Raise a symbolic expression or constructor value to a power.

Parameters

```
pow(  
  base,  
  exponent  
) -> bytes
```

6.3 Printing and inspection

6.3.1 print-settings

Create Spenso printer settings.

"typst" uses valid Typst math syntax with indexed tensor notation; "compact" uses Spenso's concise Symbolica-style form. Any field may be overridden without changing the others.

Parameters

```
print-settings(
  preset: str,
  with-dim: bool none,
  parens: bool none,
  commas: bool none,
  index-subscripts: bool none,
  symbol-scripts: bool none
) -> dictionary
```

6.3.1.0.1 preset str

Base preset: "typst" or "compact".

Default: "typst"

6.3.1.0.2 with-dim bool or none

Include representation dimensions in printed slots.

Default: none

6.3.1.0.3 parens bool or none

Use parentheses around tensor arguments.

Default: none

6.3.1.0.4 commas bool or none

Separate tensor arguments with commas.

Default: none

6.3.1.0.5 index-subscripts bool or none

Render indices as subscripts where supported.

Default: none

6.3.1.0.6 symbol-scripts bool or none

Render non-slot tensor arguments as symbol scripts where supported.

Default: none

6.3.2 to-typst-source

Print an expression as Typst math source using Spenso's printer.

Parameters

```
to-typst-source(
  expression,
  settings: dictionary
) -> str
```

6.3.2.0.1 expression

Atom payload or constructor value.

6.3.2.0.2 settings dictionary

Settings created by print-settings.

Default: `_print-settings()`

6.3.3 to-typst

Print and evaluate an expression as Typst math content.

Parameters

```
to-typst(
  expression,
  settings: dictionary,
  block: bool
) -> content
```

6.3.3.0.1 expression

Atom payload or constructor value.

6.3.3.0.2 settings dictionary

Settings created by print-settings.

Default: `_print-settings()`

6.3.3.0.3 block bool

Display the result as a block equation.

Default: `false`

6.3.4 to-string

Print an expression in Spenso's compact Symbolica notation.

Parameters

```
to-string(
  expression,
  settings: dictionary
) -> str
```


6.3.4.0.1 expression

Atom payload or constructor value.

6.3.4.0.2 settings `dictionary`

Settings created by `print-settings`.

Default: `_print-settings(preset: "compact")`

6.3.5 inspect

Decode an Atom payload into a recursive CBOR expression tree.

The result uses kind values such as `symbol`, `number`, `function`, `sum`, `product`, and `power`. Function nodes include their name, arguments, and symmetry flags. This view is ordinary Typst data; transforming the expression still uses its lossless Atom payload.

Parameters

`inspect(expression)` -> `dictionary`

6.4 Metric and index transformations

6.4.1 cook-function

Cook a tensor function into Idenso's canonical structure.

Parameters

`cook-function(expression)` -> `bytes`

6.4.2 cook-indices

Cook indices into Idenso's canonical structure.

Parameters

`cook-indices(expression)` -> `bytes`

6.4.3 expand-bis

Selectively expand bispinor structures.

Parameters

`expand-bis(expression)` -> `bytes`

6.4.4 expand-metrics

Expand all supported metric structures.

Parameters

`expand-metrics(expression)` -> `bytes`

6.4.5 expand-mink

Selectively expand Minkowski structures.

Parameters

```
expand-mink(expression) -> bytes
```

6.4.6 expand-mink-bis

Selectively expand combined Minkowski and bispinor structures.

Parameters

```
expand-mink-bis(expression) -> bytes
```

6.4.7 list-dangling

Return the dangling indices as compatible Atom payloads.

Parameters

```
list-dangling(expression) -> array
```

6.4.8 simplify-metrics

Contract and simplify metric tensors.

Parameters

```
simplify-metrics(expression) -> bytes
```

6.4.9 to-dots

Rewrite supported contractions as dot products.

Parameters

```
to-dots(expression) -> bytes
```

6.4.10 wrap-dummies

Wrap dummy indices under the given header symbol.

Parameters

```
wrap-dummies(  
  expression,  
  header  
) -> bytes
```

6.4.11 wrap-indices

Wrap all indices under the given header symbol.

Parameters

```
wrap-indices(
  expression,
  header
) -> bytes
```

6.5 Dirac and color transformations**6.5.1 dirac-adjoint**

Take the Dirac adjoint of a tensor expression.

Parameters

```
dirac-adjoint(expression) -> bytes
```

6.5.2 expand-color

Selectively expand color structures.

Parameters

```
expand-color(expression) -> bytes
```

6.5.3 simplify-color

Simplify color tensors.

Parameters

```
simplify-color(expression) -> bytes
```

6.5.4 simplify-gamma

Simplify gamma-matrix expressions.

Parameters

```
simplify-gamma(expression) -> bytes
```

6.6 Advanced configuration**6.6.1 init**

Create an independent Tydenso API.

The returned dictionary contains tensor constructors, Spenso-aware printers, CBOR inspection, and Idenso transformations. Use a custom source only when developing another compatible plugin; ordinary documents can use the imported top-level functions.

```
#let tensors = init()
#let V = (tensors.mink)(4)
#let p = (tensors.vector)("p")
#(tensors.to-typst)(p((tensors.slot)(V, "mu")))
```

 p^μ
Parameters

```
init(
  namespace: str,
  source: str bytes none,
  grammar: dictionary
) -> dictionary
```

6.6.1.0.1 namespace `str`

Default namespace for symbols parsed from Typst math.

Default: `"spenso"`

6.6.1.0.2 source `str` or `bytes` or `none`

WebAssembly plugin path or uncompressed module bytes. `none` selects the bundled compressed Tydenso engine.

Default: `none`

6.6.1.0.3 grammar `dictionary`

Parser grammar used by math unless it receives an explicit override.

Default: `_default-grammar`

7 Compatibility and licensing

This manual describes Tydenso 0.1.0 and Typst 0.14 or newer. Tydenso's original source is released under the [MIT License](#). The bundled engine also contains Spenso, Idenso, and Symbolica; their own upstream terms continue to apply. In particular, the MIT license for this interface does not relicense Symbolica. Read [Symbolica's license](#) before redistributing or deploying the WebAssembly bundle.

8 Acknowledgements

Tydenso would not exist without [Symbolica](#), the exact algebra engine beneath its Atom representation. Thank you to Symbolica's contributors, and to the GammaLoop contributors for [Spenso](#) and [Idenso](#), whose tensor model, printers, and identities define this package's mathematics.

Thanks also to [Tidy](#), which powers the worked examples and generated reference, and to [Parsely](#), which parses Typst math while preserving the semantic Atom metadata.